

NON-PROVISIONAL PATENT APPLICATION

METHOD AND SYSTEM FOR VERIFIABLE DELAY FUNCTION- PROTECTED BATCH RANDOM NUMBER GENERATION WITH ANTI-PREVIEW GUARANTEES

可驗證延遲函數保護之批次隨機數生成方法與系統
——具反預覽保證

Inventor: [Inventor Name]

Date of Filing: [Filing Date]

Priority Date: [Provisional Filing Date]

Provisional Application No.: [Provisional Serial Number]

Prepared for: Saint Island International Patent & Law Offices

聖島國際專利商標聯合事務所

ATTORNEY-CLIENT PRIVILEGED & CONFIDENTIAL

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application claims the benefit of U.S. Provisional Application No. [Provisional Serial Number], filed [Provisional Filing Date], entitled "Method and System for Verifiable Delay Function-Protected Batch Random Number Generation with Anti-Preview Guarantees," which is incorporated herein by reference in its entirety.

[0002] This application is related to U.S. Patent Application Serial No. [Patent #1 Serial Number] (hereinafter "the First Application"), entitled "Method and System for Gaming Random Number Generation," filed [date], which describes a Merkle-tree-based random number generation and verification system with blockchain anchoring.

[0003] This application is also related to U.S. Patent Application Serial No. [Patent #3 Serial Number] (hereinafter "the Third Application"), entitled "Method and System for Generating, Scoring, Signing, Anchoring, and Verifying Verifiable Entropy Objects," filed [date], which describes a structured, self-describing, cryptographically verifiable entropy object format with multi-dimensional quality scoring.

[0004] The present invention provides a complementary guarantee to the First Application. Where the First Application provides post-commitment integrity through Merkle tree verification (proving that random values were not changed after the root was committed), the present invention provides pre-commitment unpredictability through Verifiable Delay Function computation (proving that random values could not have been known before the commitment was made). Together, these two applications provide complete temporal integrity — values are neither knowable before generation nor alterable after generation.

[0005] The present invention further provides the Trust Source layer (Layer 1) of the Computational Trust Infrastructure described in the related applications, generating the

foundational entropy material that the Third Application's Verifiable Entropy Objects encapsulate and attest.

FIELD OF THE INVENTION

[0006] The present invention relates generally to systems and methods for generating cryptographically secured random numbers. More specifically, the invention relates to a pipeline integrating Verifiable Delay Functions, distributed randomness beacons, multi-source entropy aggregation, deterministic batch generation, Merkle tree construction, immutable ledger anchoring, and pool-based token serving — providing verifiable anti-preview guarantees that prevent any party, including the system operator, from previewing, predicting, pre-computing, or selectively filtering random values before the batch generation process is complete.

BACKGROUND OF THE INVENTION

1. The Insider Preview Vulnerability in Batch Random Number Generation

[0007] In many applications — online gaming, AI agent task allocation, clinical trial randomization, financial algorithm execution, lottery systems, and autonomous decision-making — random numbers must be generated in advance, stored in pools, and served on demand at very low latency (often sub-2 milliseconds). This batch-and-pool architecture is necessary because real-time random number generation cannot meet the latency requirements of high-throughput applications.

[0008] Merkle-tree-based verification systems (such as those described in the First Application) provide an important integrity guarantee for batch random number systems: once the

Merkle root hash is computed and anchored to an immutable ledger, any modification to any leaf value in the tree is cryptographically detectable. This provides *post-commitment integrity* — the guarantee that values were not tampered with after the batch was committed.

[0009] However, Merkle-tree verification alone does not address a distinct and equally critical vulnerability: the *insider preview attack*. In this attack:

- a. The system operator generates one or more candidate batches of N random values and constructs Merkle trees for each.
- b. Before anchoring any root hash to the immutable ledger, the operator previews all values in all candidate batches.
- c. The operator selects which batch to anchor, choosing a batch whose values are favorable to the operator's interests (or unfavorable to specific consumers, competitors, or counterparties).
- d. The selected batch passes all Merkle verification checks because the values were never modified after commitment — the attack does not alter values but rather selects among pre-computed alternatives.

[0010] The insider preview attack is particularly dangerous because it is undetectable by post-commitment verification alone. The Merkle tree is cryptographically intact. The blockchain anchor is genuine. All verification checks pass. The manipulation occurred in the temporal gap between value generation and commitment — a gap that Merkle verification, by design, does not cover.

2. Limitations of Existing Approaches

[0011] Hardware Random Number Generators (HRNGs). HRNGs produce entropy from physical processes (thermal noise, radioactive decay, quantum phenomena). While the

entropy quality may be excellent, HRNGs provide no mechanism to prove that the operator did not preview, buffer, filter, or selectively discard the output before incorporating it into a batch. The physical entropy generation is not time-locked; values are available instantly upon generation, enabling immediate preview.

[0012] Pseudo-Random Number Generators (PRNGs) and Cryptographically Secure PRNGs (CSPRNGs). PRNGs and CSPRNGs are deterministic given a seed. If the seed is known — or can be determined — before batch generation, the entire batch is immediately computable and therefore previewable. Even CSPRNGs that use high-quality seeds (e.g., from `/dev/urandom`) do not prevent the operator from computing and previewing the full batch instantaneously, because the computation is fast and non-sequential.

[0013] Verifiable Random Functions (VRFs). VRFs, such as those described in U.S. Patent No. 12,401,532 B2 (Chainlink, "Method and Apparatus for Producing Verifiable Randomness Within a Decentralized Computing Network"), prove that a specific output was correctly derived from a specific input using a specific secret key. However, VRFs have several limitations in the context of anti-preview batch generation:

- a. VRFs are typically designed for single-value generation, not batch generation of thousands of values for pool-based serving.
- b. The holder of the VRF secret key can compute the VRF output for any input instantaneously, enabling preview of future outputs given anticipated future inputs.
- c. VRFs do not inherently introduce a time-lock: the VRF computation is fast (typically sub-millisecond), allowing rapid generation and evaluation of multiple candidate outputs.
- d. VRFs require an on-chain callback mechanism for verification, preventing offline or air-gapped verification.

e. VRFs typically use a single entropy source (the secret key holder's computation), not multi-source aggregation.

[0014] Commit-Reveal Schemes. Traditional commit-reveal protocols require interactive participation from multiple parties: one party commits to a value (publishing a hash), and later reveals the value. While this prevents single-party manipulation, commit-reveal schemes require real-time coordination, introduce latency equal to the reveal delay, require all parties to be online simultaneously, and do not scale to batch generation of tens of thousands of values for pool-based sub-millisecond serving.

[0015] Trusted Execution Environments (TEEs). TEEs (Intel SGX, ARM TrustZone, AWS Nitro Enclaves) provide hardware-isolated execution that prevents the host from inspecting computation. However, TEEs prove execution isolation, not temporal unpredictability. A TEE can faithfully execute a batch generation algorithm, but the TEE attestation does not prove that the values were unknowable before a specific time — only that the execution environment was not tampered with. TEE-based approaches also introduce hardware dependency and supply-chain trust assumptions.

[0016] Public Randomness Beacons (e.g., drand, NIST Randomness Beacon). Public beacons produce verifiable random values at regular intervals. However, beacons produce single values per round (not batches), operate at fixed intervals (typically 3-30 seconds, too slow for per-request serving), and do not address the batch-to-pool pipeline required for sub-millisecond serving.

3. Verifiable Delay Functions — Prior Art and Distinction

[0017] Verifiable Delay Functions (VDFs) are a class of cryptographic primitive formally defined by Boneh, Bonneau, Bünz, and Fisch ("Verifiable Delay Functions," IACR ePrint 2018/601, 2018). A VDF takes a prescribed amount of sequential computation time T to

evaluate, even on a parallel computer, and produces an output accompanied by a proof that can be efficiently verified (typically in $O(\log T)$ time).

[0018] VDFs have been proposed and implemented in several contexts:

- a. **Boneh et al. (2018)**: Theoretical definition of VDFs, including Wesolowski and Pietrzak constructions based on repeated squaring in groups of unknown order.
- b. **Tezos (Nomadic Labs, 2022)**: Integration of VDFs into the Kathmandu protocol upgrade for improving random seed generation for block proposer selection in proof-of-stake consensus.
- c. **StarkWare VeeDo**: A VDF computation service with STARK proof verification on Ethereum, demonstrating VDF + on-chain verification.
- d. **Ethereum Research**: Proposals for VDF-based randomness in Ethereum's consensus layer.

[0019] While the VDF primitive itself is known, no prior art describes the specific combination of innovations in the present invention: (i) integration of VDFs into a batch random number generation pipeline producing tens of thousands of values from a single VDF output, (ii) combination of VDF anti-preview with Merkle anti-tamper to provide dual temporal integrity, (iii) asynchronous blockchain anchoring decoupled from token availability, (iv) multi-worker VDF computation with priority-based scheduling, (v) shared and dedicated pool architectures with automatic refill, (vi) hierarchical entropy source fallback with transparent provenance recording, and (vii) integration with distributed randomness beacons for publicly verifiable VDF input seeds.

SUMMARY OF THE INVENTION

[0020] The present invention provides a method and system for generating batches of verifiably unpredictable random numbers with anti-preview guarantees. The invention addresses the insider preview vulnerability in batch random number generation systems by introducing a Verifiable Delay Function (VDF) computation step that creates a temporal barrier, ensuring that no party — including the system operator — can know, predict, preview, filter, or selectively commit random values before a minimum sequential computation time has elapsed.

[0021] The invention provides:

1. A VDF-protected seed generation pipeline with configurable computation time T , providing a mathematically guaranteed anti-preview barrier.
2. Multi-source entropy aggregation combining distributed randomness beacons, blockchain entropy, and optionally hardware entropy sources, with automatic hierarchical fallback and transparent provenance recording.
3. Deterministic batch derivation producing N random values (where N may be 1,024 to 1,048,576 or more) from a single VDF output, organized in a Merkle tree for individual-value verification.
4. Dual temporal integrity guarantees: VDF-based anti-preview (values unknowable before T) combined with Merkle-based anti-tamper (values unalterable after commitment).
5. Asynchronous anchoring architecture decoupling token availability from immutable ledger confirmation, enabling sub-millisecond token serving while anchoring proceeds in the background.
6. Multi-worker VDF computation enabling parallel generation of multiple batches while maintaining sequential (anti-preview) integrity within each batch.

7. Shared and dedicated pool architectures for different deployment scenarios, with automatic threshold-based and emergency refill mechanisms.
 8. Integration with Verifiable Entropy Object (VEO) standards for structured trust evidence packaging.
-

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 illustrates the overall system architecture of the VDF-protected batch random number generation system.

FIG. 2 illustrates the VDF anti-preview timeline showing the temporal barriers provided by VDF computation, Merkle commitment, and blockchain anchoring.

FIG. 3 illustrates the batch derivation pipeline from entropy acquisition through VDF computation, batch generation, Merkle tree construction, and dual-path output.

FIG. 4 illustrates the multi-worker VDF architecture with priority-based scheduling.

FIG. 5 illustrates the shared pool architecture with pre-warming, threshold-based refill, and per-consumer rate limiting.

FIG. 6 illustrates the hierarchical entropy source architecture with automatic fallback and provenance recording.

FIG. 7 illustrates the complete token lifecycle from VDF-protected generation through pool serving, consumer delivery, and independent verification.

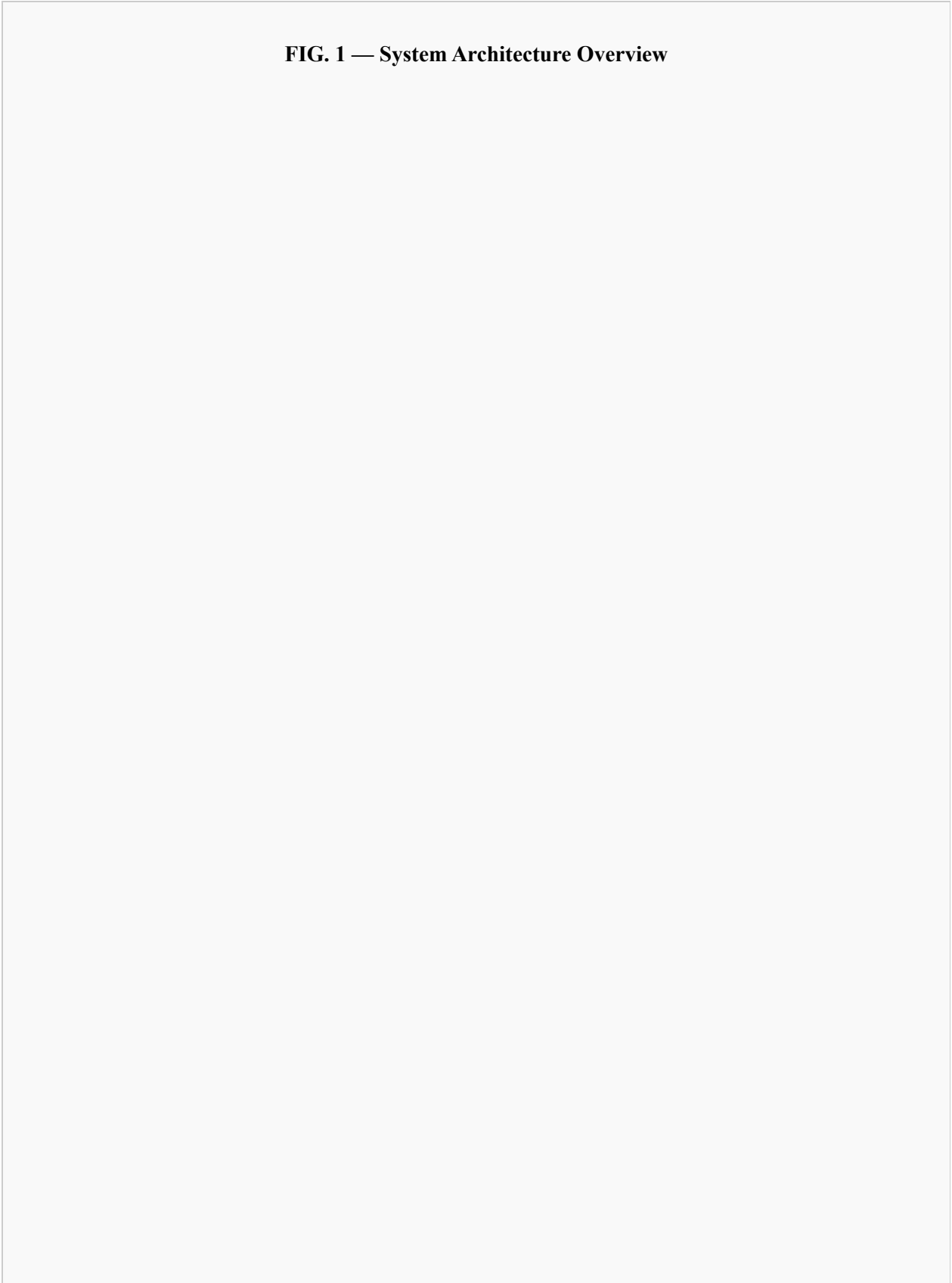
DETAILED DESCRIPTION OF PREFERRED EMBODIMENTS

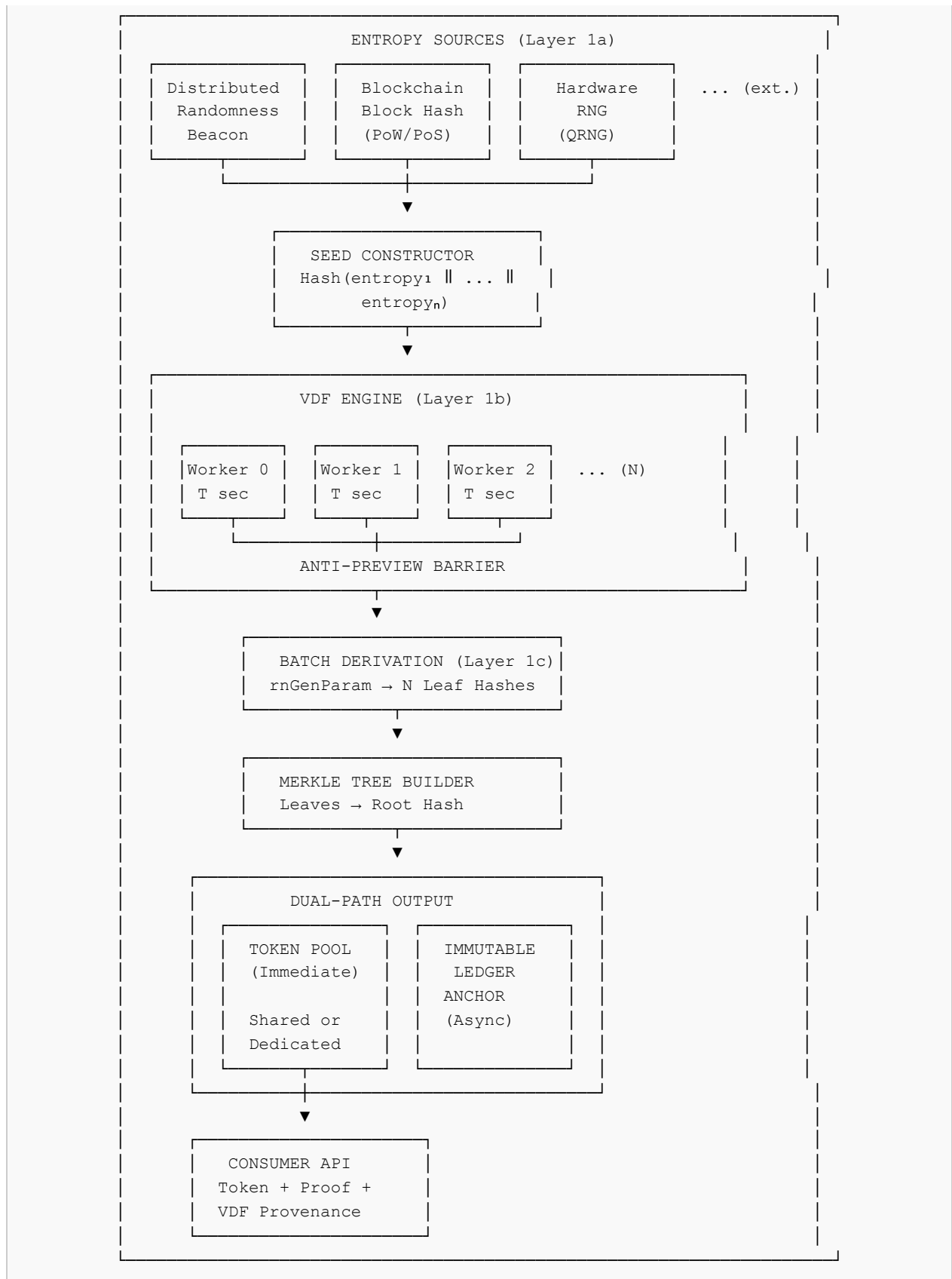
[0022] The following description sets forth specific embodiments of the invention. These embodiments are illustrative and not limiting; the invention encompasses all equivalents, variations, and modifications within the scope of the appended claims.

1. System Architecture Overview

[0023] Referring to FIG. 1, the VDF-protected batch random number generation system comprises the following major components interconnected as described herein:

FIG. 1 — System Architecture Overview





2. VDF-Protected Seed Generation Pipeline

[0024] The core innovation of the present invention is the VDF-protected seed generation pipeline, which creates a temporal barrier ensuring that no party can preview the random values before a minimum sequential computation time T has elapsed. The pipeline comprises the following steps:

2.1 Entropy Acquisition

[0025] The system obtains entropy from one or more independent sources arranged in a hierarchical fallback architecture (FIG. 6). In the preferred embodiment, the hierarchy is:

[0026] **Tier 1 — Distributed Randomness Beacon.** The preferred entropy source is a distributed randomness beacon, such as the drand network operated by the League of Entropy (comprising Cloudflare, Protocol Labs, academic institutions, and other independent operators). The drand beacon produces a new random value approximately every 3 seconds. Each beacon round provides: (a) a round number, (b) a randomness value (32 bytes, hexadecimal-encoded), and (c) a BLS threshold signature. The system verifies the beacon's integrity by confirming that $\text{SHA-256}(\text{signature})$ equals the published randomness value. In alternative embodiments, other distributed beacons may be used, including the NIST Randomness Beacon, Ethereum's RANDAO, or any future publicly verifiable randomness service.

[0027] **Tier 2 — Blockchain Block Entropy.** If the distributed beacon is unavailable (network failure, timeout exceeding a configurable threshold, integrity check failure, or consecutive failures exceeding a configurable maximum), the system obtains entropy from one or more blockchain block hashes. Block hashes from proof-of-work chains (e.g., Bitcoin) provide entropy derived from the computational effort of the mining network. Block hashes from proof-

of-stake chains (e.g., Ethereum, Polygon) provide entropy derived from validator consensus. Multiple block hashes from independent chains may be combined for increased source diversity.

[0028] Tier 3 — Local Cryptographic Entropy. As a final fallback, the system uses operating system cryptographic randomness (e.g., `/dev/urandom`, `crypto.randomBytes()`, or equivalent) combined with system-specific entropy (timestamps, process identifiers, hardware counters). This tier provides the lowest source diversity but maintains the anti-preview guarantee because the VDF computation time is independent of the entropy source.

[0029] Provenance Recording. The entropy source tier used for each batch is recorded in the batch metadata. This transparent provenance enables consumers and auditors to distinguish between batches sourced from publicly verifiable beacons (Tier 1), blockchain-derived entropy (Tier 2), and locally-sourced entropy (Tier 3). There are no silent fallbacks — every degradation in source quality is visible in the metadata.

2.2 Seed Construction

[0030] The system constructs a VDF input seed by combining entropy from the selected tier with additional entropy contributions using one or more cryptographic hash functions. In the preferred embodiment:

$$\text{vdfSeed} = H(\text{blockParam} \parallel \text{randomEntropy} \parallel \text{additionalEntropy})$$

[0031] where H is a cryptographic hash function (SHA-256 in the preferred embodiment; SHA-3, BLAKE2, BLAKE3, or other cryptographic hash functions in alternative embodiments), `blockParam` is derived from a blockchain block hash, `randomEntropy` is the beacon or local entropy value, and `additionalEntropy` is optional additional randomness (e.g.,

from a hardware random number generator). The concatenation operator $\parallel$ denotes bitstring concatenation.

[0032] The use of multiple entropy contributions ensures that no single source controls the VDF input seed. Even if one entropy source is compromised or predictable, the remaining sources contribute unpredictability to the seed.

2.3 VDF Computation

[0033] The VDF input seed is processed through a Verifiable Delay Function (VDF) requiring a minimum sequential computation time of T seconds. The VDF computation is the anti-preview barrier: no party — including the system operator, the entropy source operators, and any external attacker — can determine the VDF output before T seconds of sequential computation have elapsed.

[0034] In the preferred embodiment, the VDF is implemented as an iterative sequential hash chain:

$$\begin{aligned}h[0] &= H(\text{seed} \parallel "0") \\h[1] &= H(h[0] \parallel "1") \\h[2] &= H(h[1] \parallel "2") \\&\dots \\h[K] &= H(h[K-1] \parallel "K") = \text{vdfOutput}\end{aligned}$$

[0035] where $K = T \times C$ (T being the target computation time in seconds, C being the number of hash operations per second for the deployment hardware — default $C = 10,000$ in the preferred embodiment). Each iteration depends on the previous iteration's output, creating a strict sequential dependency that prevents parallelization. Even with unlimited parallel processors, the computation cannot complete in fewer than K sequential hash operations.

[0036] The VDF produces two outputs: (a) the VDF output value (the final hash $h[K]$), and (b) a VDF proof enabling efficient verification. In the preferred embodiment, the proof is computed as $H(\text{vdfOutput} \parallel \text{seed})$. In alternative embodiments, the VDF may be implemented using:

- a. **Wesolowski VDF**: Repeated squaring in a group of unknown order (e.g., RSA group), with a succinct proof verifiable in $O(\log T)$ time.
- b. **Pietrzak VDF**: An alternative construction with a larger proof size but potentially faster verification.
- c. **Sloth function**: Modular square root computation with inherent sequentiality.
- d. **Any future VDF construction** that provides the sequential computation property and efficient verifiability.

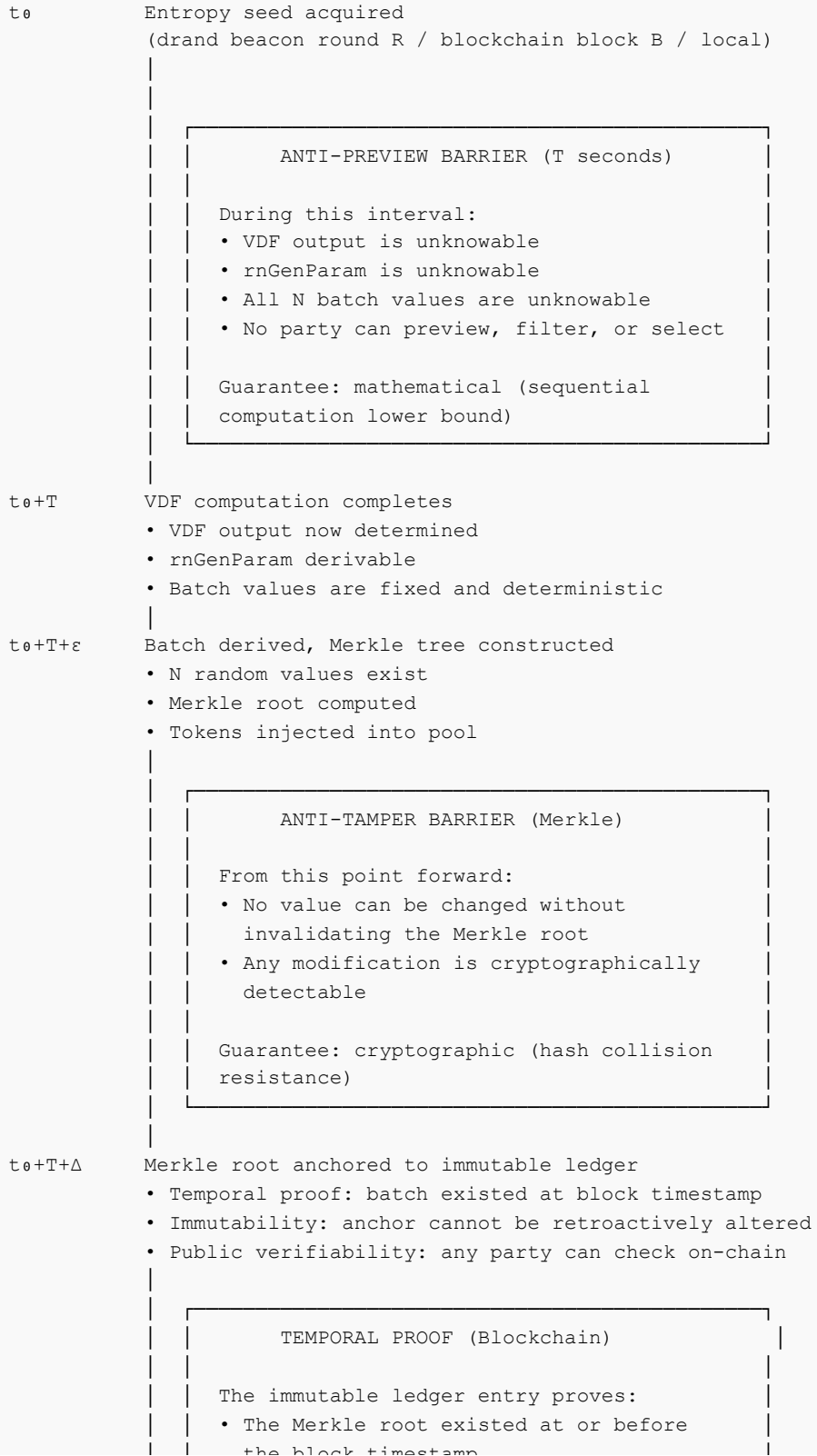
[0037] The configurable parameter T allows the system to balance anti-preview security against batch generation throughput. Higher T values provide stronger anti-preview guarantees (longer windows during which values are unknowable) but reduce the rate at which new batches can be generated. In the preferred embodiment, T defaults to 4 seconds, which is sufficient to prevent real-time preview attacks while enabling multiple batches per minute via multi-worker parallelism.

2.4 The Anti-Preview Property

[0038] Referring to FIG. 2, the VDF computation time T creates a temporal barrier with the following property:

FIG. 2 — Dual Temporal Integrity Timeline

TIMELINE



		the block timestamp • The commitment is permanent	
COMPLETE TEMPORAL INTEGRITY:			
	Before t_0+T :	Values UNKNOWABLE	(VDF guarantee)
	After $t_0+T+\epsilon$:	Values UNALTERABLE	(Merkle guarantee)
	After $t_0+T+\Delta$:	Values TIMESTAMPED	(Blockchain guarantee)

[0039] This dual temporal integrity — anti-preview before commitment plus anti-tamper after commitment — is a fundamental contribution of the present invention. No prior art provides both guarantees in a single batch generation system.

3. Deterministic Batch Derivation

[0040] From the VDF output, the system deterministically generates N random values through the following process:

[0041] Step 1 — RN Generation Parameter. The VDF output is combined with the block parameter to produce the RN (Random Number) Generation Parameter:

```
rnGenParam = H(vdfOutput || blockParam)
```

[0042] Step 2 — Leaf Hash Generation. For each leaf index i ($0 \leq i < N$), the system generates a unique node identifier and computes a leaf hash:

```
nodeId[i] = formatIdentifier(i, batchId)
leafHash[i] = H(rnGenParam || nodeId[i])
```

[0043] Step 3 — Numeric Value Derivation. Each leaf hash is converted to a numeric random value by extracting a prefix of the hash and mapping it to the target range. In the preferred embodiment, the first 8 hexadecimal characters are extracted and divided by 0xFFFFFFFF, producing a uniformly distributed floating-point value in the range [0, 1). For

integer ranges, the value is scaled and truncated: $\text{intValue} = \text{floor}(\text{floatValue} \times (\text{max} - \text{min} + 1)) + \text{min}$.

[0044] The batch size N is configurable. In the preferred embodiment, $N = 65,536$ (2^{16}). In alternative embodiments, N may range from 1,024 (2^{10}) for low-throughput applications to 1,048,576 (2^{20}) or more for high-throughput deployments. The derivation is fully deterministic: given the same `rnGenParam` and batch metadata, any party can independently regenerate the identical batch.

4. Merkle Tree Construction and Verification

[0045] The N leaf hashes are organized into a binary Merkle tree. Adjacent leaves are concatenated and hashed pairwise to produce parent nodes, recursively until a single root hash is obtained. When the number of leaves at any level is odd, the last leaf is duplicated to form a complete pair.

[0046] Each individual token can be verified against the Merkle root using a proof path of $O(\log_2 N)$ hash comparisons. For $N = 65,536$, this requires 16 hash comparisons — computationally trivial. The verification algorithm traverses the proof path, combining the leaf hash with sibling hashes at each level, and confirms that the final computed value equals the published root hash.

[0047] Verification requires no cooperation from the system operator. Any party with access to: (a) the leaf hash, (b) the proof path, (c) the on-chain root hash, and (d) optionally the VDF provenance metadata, can independently verify the token. This verification can be performed offline, air-gapped, and at any future time.

5. Asynchronous Anchoring Architecture

[0048] A key architectural innovation is the decoupling of token availability from immutable ledger confirmation. In conventional systems, tokens cannot be served until the blockchain transaction confirming the Merkle root is finalized — introducing latency of seconds to minutes. The present invention decouples these operations:

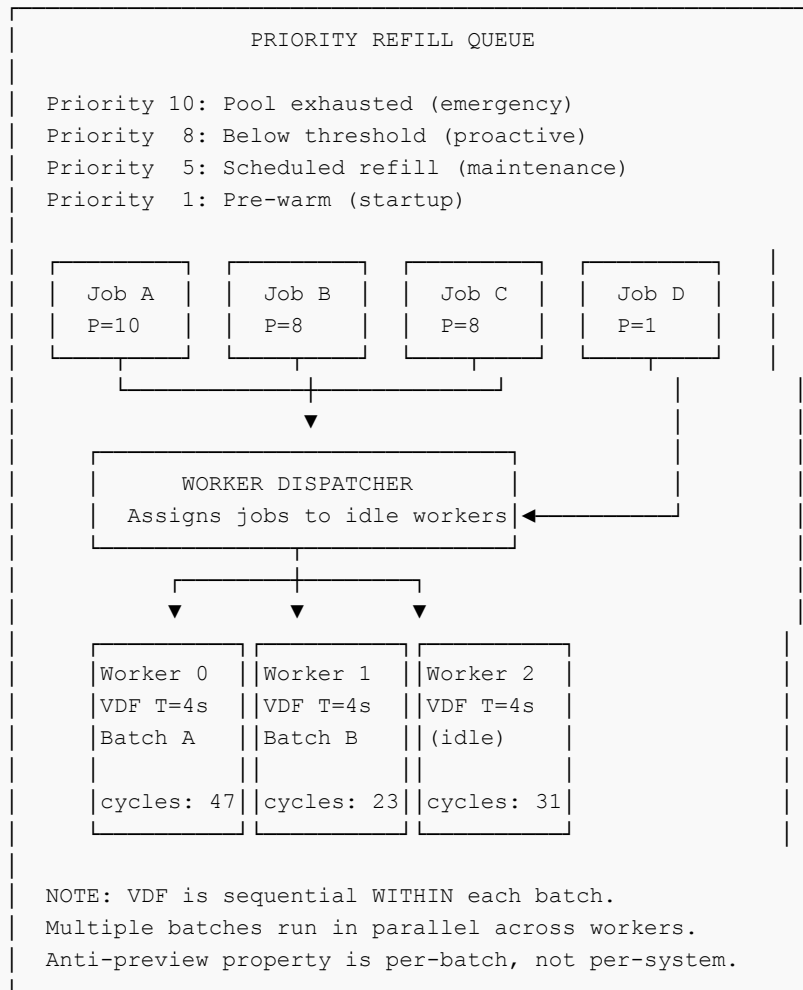
- a. **Immediate injection:** Tokens are injected into the serving pool immediately upon batch derivation, before blockchain anchoring. The anti-preview guarantee has already been provided by the VDF computation; the Merkle tree provides anti-tamper; the blockchain anchor adds temporal proof and immutability but is not required for the anti-preview or anti-tamper properties.
- b. **Asynchronous anchoring:** The Merkle root is submitted to the immutable ledger (blockchain smart contract, in the preferred embodiment) in a background process with configurable retry logic (default: 3 attempts with exponential backoff).
- c. **Status tracking:** Each token carries an anchor status field that transitions through states: "pending" (submitted but not confirmed), "confirmed" (transaction confirmed at specific block), "failed" (all retry attempts exhausted). The status is queryable by consumers.
- d. **Retroactive upgrade:** Upon anchor confirmation, all tokens in the batch are retroactively upgradeable to the highest verification level (anchored verification) without regeneration.

[0049] In the preferred embodiment, the immutable ledger is a smart contract on the Polygon blockchain (contract address: 0xA79E149C35Ad47Ed270Bf4b16B80170eBF7B88F8 on Polygon Amoy testnet). In alternative embodiments, the immutable ledger may be any blockchain (Ethereum, Bitcoin via OP_RETURN, Solana, etc.), any append-only ledger, any transparency log, or any distributed timestamping system that provides immutability and public verifiability.

6. Multi-Worker VDF Architecture

[0050] Referring to FIG. 4, the system employs multiple VDF worker threads to enable concurrent generation of multiple batches. Each worker is an independent computation unit capable of running one VDF computation at a time.

FIG. 4 — Multi-Worker VDF Architecture



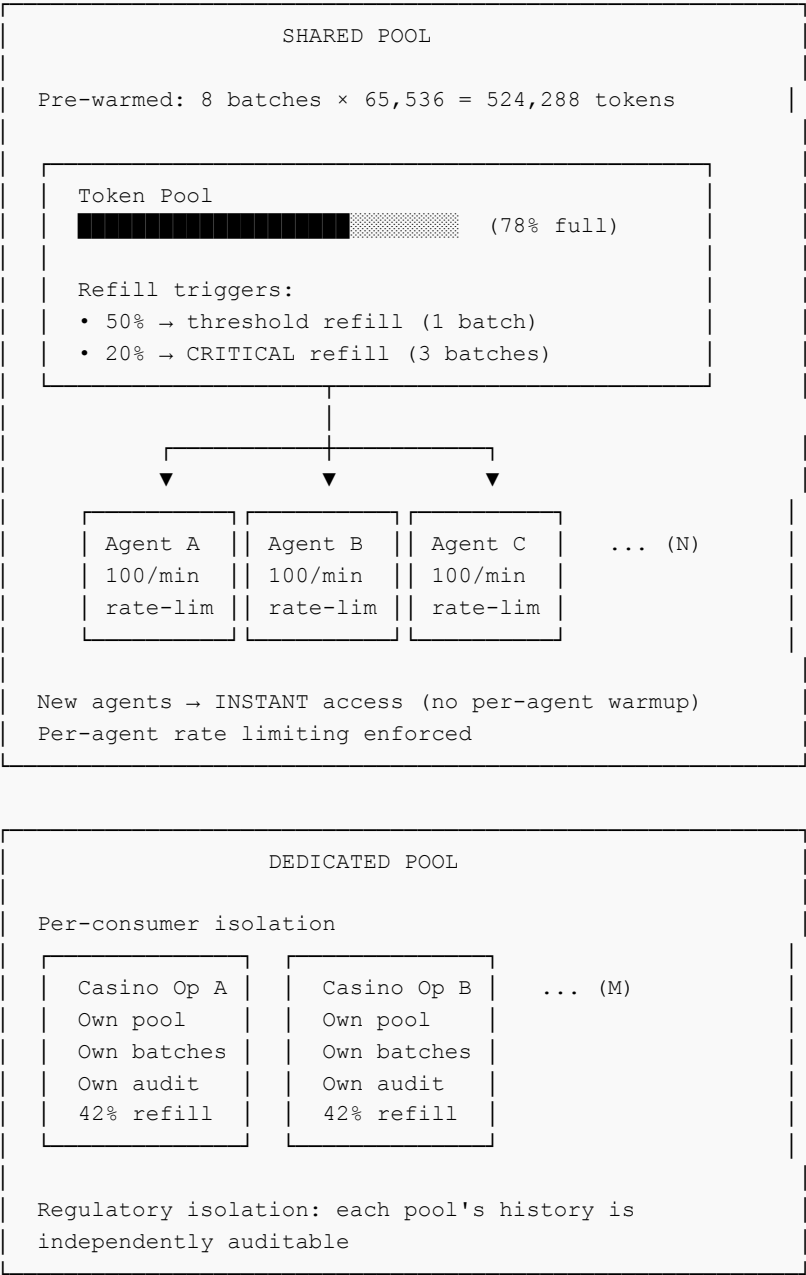
[0051] In the preferred embodiment, the system deploys 3 workers by default. Each worker maintains state: worker ID, busy/idle status, current batch ID, assigned client ID, and completed cycle count. The dispatcher assigns queued jobs to idle workers in priority order.

[0052] The critical property maintained by this architecture: the VDF's sequential computation guarantee applies *within each batch*. A single batch's values cannot be determined faster by adding workers. Workers enable higher system-wide throughput (multiple batches per VDF period) without weakening the per-batch anti-preview guarantee.

7. Shared Pool and Dedicated Pool Architectures

[0053] Referring to FIG. 5, the system supports two complementary pool modes:

FIG. 5 — Shared Pool Architecture



[0054] Shared Pool. A system-wide pool serves all consumers in designated verticals (e.g., AI agents, general-purpose applications). The shared pool is pre-warmed at startup with a

configurable number of VDF-protected batches (default: 8 batches = 524,288 tokens). Consumers access the shared pool with per-consumer rate limiting (configurable; default: 100 requests per minute). New consumers can begin consuming tokens immediately — no per-consumer batch generation delay. Refill is triggered at configurable thresholds: 50% for standard refill (one batch), 20% for critical multi-batch emergency refill (up to 3 concurrent batches).

[0055] Dedicated Pool. Per-consumer pools provide isolation for verticals requiring independent audit trails (e.g., regulated gaming, clinical trials). Each consumer has an independent pool with independent refill triggers, batch history, and audit trail. Dedicated pools are pre-warmed with one batch upon consumer registration.

8. Token Serving and Proof Delivery

[0056] Referring to FIG. 7, when a consumer requests one or more random tokens, the system:

- a. Checks rate limits (for shared pool consumers).
- b. Consumes the requested number of tokens from the appropriate pool (shared or dedicated) using a sequential pointer ($O(1)$ retrieval).
- c. For each consumed token, generates a Merkle proof path from the token's leaf hash to the batch's Merkle root.
- d. Packages the response including: the random value, the leaf hash, the node identifier, the batch identifier, the complete Merkle proof path, the Merkle root, the VDF provenance metadata (entropy source, VDF computation time, VDF proof), and the blockchain anchor reference (transaction hash and block number, or "pending" if not yet confirmed).
- e. Records the consumption event (consumer identifier, timestamp, token identifiers).

f. Checks pool depth and triggers refill if below threshold.

[0057] The entire serving process operates at sub-2-millisecond latency because all data is served from in-memory structures. No VDF computation, no blockchain query, and no external service call occurs during serving — those operations happened during batch generation (VDF) and background anchoring (blockchain).

9. Independent Verification

[0058] Any party may independently verify a served token by performing the following checks, none of which requires cooperation from the system operator:

- a. **Integrity check:** Recompute $H(\text{rnGenParam} \parallel \text{nodeId})$ and confirm it equals the claimed leaf hash.
- b. **Merkle proof check:** Traverse the proof path, combining the leaf hash with sibling hashes at each level, and confirm the computed root equals the published Merkle root.
- c. **Blockchain anchor check:** Query the on-chain smart contract to confirm the published Merkle root is recorded at the claimed transaction hash and block number.
- d. **VDF provenance check:** Verify the VDF proof against the VDF seed, confirming the VDF computation was performed correctly.
- e. **Entropy source check:** If the entropy source was a distributed beacon (Tier 1), independently fetch the beacon round and verify its BLS signature against the published chain public key.

[0059] These checks can be performed offline (except the blockchain anchor check, which requires network access or a local chain node), at any future time, and without any interaction with the system operator.

10. Alternative Embodiments

10.1 Alternative VDF Constructions

[0060] While the preferred embodiment uses an iterative sequential hash chain, alternative embodiments may employ any VDF construction that provides the sequential computation property, including but not limited to: Wesolowski VDF (repeated squaring in RSA groups), Pietrzak VDF (halving protocol), sloth functions (modular square roots), or any future VDF construction. The choice of VDF construction does not affect the system architecture or the anti-preview guarantee — it affects only the verification efficiency and proof size.

10.2 Alternative Hash Functions

[0061] While the preferred embodiment uses SHA-256, alternative embodiments may use any cryptographic hash function including SHA-3 (Keccak), BLAKE2, BLAKE3, or future hash functions. The system architecture is hash-function-agnostic; the hash function is a configurable parameter.

10.3 Alternative Immutable Ledgers

[0062] While the preferred embodiment uses a Polygon blockchain smart contract, alternative embodiments may anchor Merkle roots to any immutable verification substrate including: public blockchains (Ethereum, Bitcoin, Solana, etc.), consortium chains, private chains, append-only distributed ledgers, certificate transparency logs, notarization networks, or distributed timestamping systems. The system architecture requires only that the substrate provides immutability (the recorded root cannot be retroactively altered) and public verifiability (any party can confirm the root was recorded at a specific time).

10.4 Alternative Entropy Sources

[0063] While the preferred embodiment uses drand as the primary entropy source, alternative embodiments may use any publicly verifiable entropy source including: NIST Randomness Beacon, Ethereum RANDAO, quantum random number generators (QRNGs), multiple independent randomness beacons combined, satellite-based entropy, or federated entropy networks. The system architecture is entropy-source-agnostic; new sources are added by implementing a source adapter.

10.5 Integration with Verifiable Entropy Objects

[0064] In an embodiment integrated with the Third Application (VEO patent), the VDF-protected batch generation pipeline serves as the Trust Source layer (Layer 1) of the Computational Trust Infrastructure. Each batch's provenance metadata — entropy source, VDF computation parameters, VDF proof, Merkle root, and blockchain anchor — is encapsulated in a Verifiable Entropy Object (VEO-1), providing a standardized, self-describing, independently verifiable trust evidence package. The VEO's Entropy Confidence Score (ECS) reflects the VDF-protected batch's trust properties: source diversity (multiple entropy contributions to the VDF seed), manipulation resistance (VDF temporal barrier), and verification success (VDF proof validity and beacon signature verification).

10.6 AI Agent Decision Attestation

[0065] In an embodiment for AI agent systems, VDF-protected random tokens are used to back autonomous agent decisions — tool selection, task routing, resource allocation, response sampling. Each decision is linked to the specific token (and its VDF provenance) that influenced it, creating an auditable decision trail. The VDF anti-preview guarantee ensures that the agent (or its operator) could not have previewed the random input to a decision before the decision was made, preventing selective manipulation of agent behavior.

10.7 Regulated Gaming

[0066] In an embodiment for regulated gaming (casinos, lotteries, online gambling), VDF-protected batches serve as the random number source for game outcomes. The dual temporal integrity guarantee — VDF anti-preview plus Merkle anti-tamper — satisfies the requirements of gaming regulators (GLI-19, GLI-33, BMM, eCOGRA) for provably fair randomness. Dedicated pools per operator provide independent audit trails. The VDF computation time T is configurable per regulatory jurisdiction.

10.8 On-Demand Generation Fallback

[0067] In an embodiment handling pool exhaustion, the system generates random values on-demand when the pool is depleted. On-demand generation bypasses the full VDF pipeline (using direct cryptographic randomness) to avoid latency, but flags the tokens with reduced provenance metadata indicating on-demand generation. Simultaneously, the system triggers emergency pool refill via the VDF pipeline. This fallback ensures zero downtime while transparently indicating that on-demand tokens do not carry the full VDF anti-preview guarantee.

CLAIMS

What is claimed is:

- 1.** A computer-implemented method for generating a batch of random numbers with anti-preview guarantees, comprising:
 - (a) obtaining one or more entropy values from at least one independent entropy source;
 - (b) constructing a seed value by combining the one or more entropy values using one or more cryptographic hash functions;

(c) computing a delay function output by processing the seed value through a delay function requiring a minimum sequential computation time T , wherein the delay function output is not determinable by any party, including a system operator, until said sequential computation time T has elapsed;

(d) deriving a generation parameter from the delay function output using one or more cryptographic hash functions;

(e) deterministically generating a plurality of N random values from the generation parameter, wherein each random value is derived as a cryptographic hash of the generation parameter combined with a unique per-value identifier;

(f) organizing the plurality of N random values as leaf nodes of a hash tree and computing a root hash; and

(g) recording the root hash on an immutable verification substrate,

wherein the delay function computation time T provides an anti-preview guarantee that no party can determine any of the plurality of random values before the delay function computation completes, and the hash tree provides an anti-tamper guarantee that no random value can be altered after the root hash is computed.

2. The method of claim 1, wherein the delay function comprises a Verifiable Delay Function (VDF) producing a proof that enables verification of the computation in time substantially less than T .

3. The method of claim 1, wherein the delay function comprises an iterative sequential hash chain in which each iteration computes a cryptographic hash dependent on the previous iteration's output, thereby preventing parallel computation.

4. The method of claim 1, wherein the at least one independent entropy source comprises a distributed randomness beacon producing publicly verifiable random values using

threshold signatures, and wherein the entropy value obtained from the beacon is verified by confirming that a cryptographic hash of the beacon's signature equals the beacon's published randomness value.

5. The method of claim 4, further comprising a hierarchical fallback mechanism wherein:

(i) if the distributed randomness beacon is unavailable, obtaining entropy from one or more blockchain block hashes; and

(ii) if blockchain entropy is also unavailable, obtaining entropy from local cryptographic randomness,

wherein the entropy source tier used for each batch is recorded in batch provenance metadata.

6. The method of claim 1, wherein step (g) of recording the root hash on an immutable verification substrate is performed asynchronously after the plurality of random values are made available for consumption, and wherein:

(i) each random value is tagged with an anchor status that transitions from a pending state to a confirmed state upon confirmation of the recording transaction; and

(ii) the anti-preview guarantee provided by the delay function is independent of the immutable verification substrate recording.

7. The method of claim 1, wherein N is at least 1,024, and wherein each individual random value is independently verifiable against the root hash using a proof path of $O(\log_2 N)$ hash comparisons without requiring cooperation from the system operator.

8. The method of claim 1, further comprising injecting the plurality of random values into a pre-generated token pool and serving individual tokens from the pool at sub-2-

millisecond latency, wherein each served token includes the random value, a hash tree proof path, a batch identifier, and delay function provenance metadata.

9. The method of claim 8, further comprising monitoring pool depth and automatically triggering generation of a new batch when the pool depth falls below a configurable threshold.

10. The method of claim 1, further comprising recording, for each batch, provenance metadata comprising at least: the entropy source identifier and tier, the delay function computation parameters, the delay function proof, the hash tree root, and the immutable verification substrate reference.

11. A system for generating verifiably unpredictable batch random numbers, comprising:

one or more processors; and

memory storing instructions that, when executed by the one or more processors, cause the system to:

- (a) obtain entropy from at least one independent entropy source;
- (b) construct a seed from the obtained entropy;
- (c) compute a delay function output from the seed, the computation requiring a minimum sequential time T ;
- (d) derive N random values deterministically from the delay function output;
- (e) organize the N random values in a hash tree and compute a root hash;
- (f) store the N random values in a token pool for serving at sub-millisecond latency; and
- (g) record the root hash on an immutable verification substrate,

wherein the delay function computation time T ensures that no party can determine any of the N random values before the delay function computation completes.

12. The system of claim 11, further comprising a plurality of delay function worker threads, each capable of computing a delay function for a separate batch concurrently, and a priority-based queue that schedules batch generation based on pool depletion urgency.

13. The system of claim 11, further comprising a shared token pool pre-warmed with a configurable number of delay-function-protected batches at system startup, wherein multiple consumers consume tokens from the shared pool with per-consumer rate limiting, and wherein new consumers access tokens immediately without per-consumer batch generation delay.

14. The system of claim 11, further comprising dedicated per-consumer token pools providing per-consumer isolation, independent batch histories, and independent audit trails.

15. The system of claim 11, wherein the instructions further cause the system to package the provenance metadata for each batch into a structured, self-describing, independently verifiable digital object comprising at least the entropy source records, delay function proof, hash tree root, and immutable verification substrate reference.

16. A computer-implemented method for providing dual temporal integrity guarantees for batch random number generation, comprising:

(a) generating a batch of N random values using a delay function pipeline that requires a minimum sequential computation time T , thereby providing an anti-preview guarantee that the values could not have been known by any party before time T elapsed;

(b) organizing the N random values into a hash tree whose root hash provides an anti-tamper guarantee that no value can be altered after the root is computed without detection; and

(c) recording the root hash on an immutable verification substrate, thereby providing a temporal proof that the values existed at or before the recording timestamp,

whereby the combination of the anti-preview guarantee, the anti-tamper guarantee, and the temporal proof provides dual temporal integrity: the values were neither knowable before generation nor alterable after generation.

17. The method of claim 16, wherein the batch of N random values is served to consumers from a pre-generated pool at sub-2-millisecond latency, and wherein each served token includes provenance metadata identifying at least the entropy source, delay function computation parameters, hash tree proof path, and immutable verification substrate reference.

18. The method of claim 16, wherein the delay function pipeline incorporates entropy from a distributed randomness beacon whose output is independently verifiable by any third party using the beacon's published cryptographic key material.

19. The method of claim 16, wherein the recording of the root hash on the immutable verification substrate is performed asynchronously after the random values are made available for consumption, and wherein the anti-preview guarantee and anti-tamper guarantee are independent of the recording completion.

20. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors, cause the one or more processors to perform a method comprising:

- (a) obtaining entropy from one or more independent sources arranged in a hierarchical fallback architecture with transparent provenance recording;
 - (b) computing a delay function output from the obtained entropy, requiring a minimum sequential computation time T ;
 - (c) deterministically deriving N random values from the delay function output;
 - (d) constructing a hash tree over the N random values;
 - (e) injecting the N random values into a token serving pool;
 - (f) asynchronously recording the hash tree root on an immutable verification substrate; and
 - (g) serving individual tokens from the pool with hash tree proofs and delay function provenance metadata at sub-millisecond latency.
-

ABSTRACT OF THE DISCLOSURE

A method and system for generating batches of random numbers with verifiable anti-preview guarantees using Verifiable Delay Functions (VDFs). The system obtains entropy from independent sources including distributed randomness beacons and blockchain block hashes through a hierarchical fallback architecture with transparent provenance recording. The entropy is processed through a VDF requiring a minimum sequential computation time T , and deterministically derives a batch of N random values from the VDF output. The batch is organized as a hash tree (Merkle tree) whose root hash is recorded on an immutable verification substrate (blockchain). The VDF computation time provides an anti-preview guarantee: no party, including the system operator, can determine the batch values before the VDF computation completes. The hash tree provides an anti-tamper guarantee: no value can be altered after the root

is computed. Together, these guarantees provide dual temporal integrity — values are neither knowable before generation nor alterable after generation. The system supports sub-2-millisecond token serving from pre-generated pools, multi-worker VDF computation for parallel batch generation, shared and dedicated pool architectures with automatic threshold-based refill, asynchronous blockchain anchoring decoupled from token availability, and integration with structured verifiable entropy object standards.